

Structures de Données et Algorithmes Avancés

Projet P3 — Heap Optimisé

Une file de priorité avec `changePriority` en $\mathcal{O}(\log n)$

Master 1 GLSI — 2025–2026

Dr. El Hadji Bassirou TOURÉ
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Contexte et objectifs

Lancement : à l'issue du Lab S04.

Rendu : avant la Séance 5 (Graphes).

Prérequis : CM et Lab S04 complétés ; Lab S01 complété (Docker, Gradle, JUnit).

Modalité : projet **individuel**.

Pourquoi ce projet ? En S05, on implémentera l'algorithme de Dijkstra pour trouver le plus court chemin dans un graphe de villes sénégalaises. Dijkstra a besoin d'une file de priorité qui permet de *modifier la priorité d'un élément déjà dans la file* — opération absente de `java.util.PriorityQueue`. Vous allez construire cette structure. **Le heap que vous livrez ici sera réutilisé tel quel dans le Projet P4 (Mazes, en S07).**

Livrables attendus

1. Une classe `OptimizedHeapMinPQ<T>` qui implémente l'interface `ExtrinsicMinPQ<T>` fournie.
2. Des tests JUnit **fournis** (à copier et lancer) **et à compléter** (au moins 3 tests supplémentaires écrits par vous).
3. Un rapport PDF court (**3 à 5 pages**) contenant : analyse de complexité de chaque opération ; *benchmark* comparatif avec `java.util.PriorityQueue` sur une séquence incluant `changePriority` ; discussion sur le choix de la structure `HashMap` interne.

Dépôt : archive zip ou lien vers un dépôt Git public (GitHub, GitLab).

Nom de l'archive : P3-Heap-PRENOM-NOM.zip

Table des matières

1	Le problème	2
1.1	Le cas de Dijkstra (avant-goût de S05)	2
1.2	Pourquoi <code>java.util.PriorityQueue</code> ne suffit pas	2
1.3	L'idée de l'optimisation	2
2	L'interface à implémenter	2
2.1	<code>ExtrinsicMinPQ<T></code> — fournie, ne pas modifier	2
2.2	La classe <code>OptimizedHeapMinPQ<T></code> — à implémenter	3

3	Ce qu'il faut faire — étape par étape	3
3.1	Étape 1 — Mettre en place le projet	3
3.2	Étape 2 — Implémenter <code>add</code> et la percolation vers le haut	4
3.3	Étape 3 — Implémenter <code>peekMin</code> , <code>removeMin</code> , <code>contains</code> , <code>size</code>	4
3.4	Étape 4 — Implémenter <code>changePriority</code>	4
4	Tests JUnit	4
4.1	Tests fournis (à copier tels quels)	4
4.2	Tests à écrire (au moins 3)	6
5	Benchmark comparatif	6
6	Rapport — contenu attendu (3 à 5 pages)	6
7	Grille d'évaluation	7
8	Règles et conseils	7
9	Remise et format du rendu	8

1 Le problème

Rappel du CM S04 : un min-heap binaire stocké dans un tableau offre `insert` et `extractMin` en $\mathcal{O}(\log n)$, et `peekMin` en $\mathcal{O}(1)$. C'est déjà très bien. Mais certains algorithmes ont besoin de plus.

1.1 Le cas de Dijkstra (avant-goût de S05)

Dijkstra cherche le plus court chemin depuis un sommet source vers tous les autres sommets d'un graphe pondéré. Le squelette de l'algorithme est :

```
initialiser dist[v] = infini pour tous les sommets, dist[source] = 0
mettre tous les sommets dans une file de priorite avec priorite = dist[v]
tant que la file n'est pas vide :
    u = extractMin de la file
    pour chaque voisin v de u :
        si dist[u] + poids(u, v) < dist[v] :
            dist[v] = dist[u] + poids(u, v)
            changePriority(v, dist[v]) <--- le point sensible
```

Le problème : à chaque *relaxation* d'une arête, on veut mettre à jour la priorité d'un sommet déjà présent dans la file. Cette opération s'appelle `changePriority(item, newPriority)`.

1.2 Pourquoi `java.util.PriorityQueue` ne suffit pas

La classe standard Java ne fournit *pas* `changePriority`. La seule solution avec cette classe est :

1. `remove(item)` — parcours linéaire en $\mathcal{O}(n)$;
2. `add(item)` avec la nouvelle priorité — $\mathcal{O}(\log n)$.

Coût total : $\mathcal{O}(n)$ par mise à jour. Pour Dijkstra qui fait E relaxations, on obtient $\mathcal{O}(E \cdot n)$, beaucoup plus lent que le $\mathcal{O}((V + E) \log V)$ possible avec un heap optimisé.

1.3 L'idée de l'optimisation

Pour que `changePriority` soit en $\mathcal{O}(\log n)$, il faut pouvoir *localiser* l'élément dans le heap en $\mathcal{O}(1)$ (au lieu de le chercher en $\mathcal{O}(n)$), puis appliquer une percolation (haut ou bas selon la nouvelle priorité) en $\mathcal{O}(\log n)$.

L'idée clé du projet

Maintenir une `HashMap` interne qui associe à chaque élément son indice courant dans le tableau du heap :

```
Map<T, Integer> itemToIndex
```

À chaque swap dans la percolation, il faut mettre à jour les deux entrées de la map. C'est l'invariant qui rend tout le projet fonctionnel.

2 L'interface à implémenter

2.1 `ExtrinsicMinPQ<T>` — fournie, ne pas modifier

```
1 import java.util.NoSuchElementException;
2
3 /**
4  * Min-priority queue dite "extrinseque" : la priorite est passee a part
5  * de l'element, et peut etre modifiee via changePriority.
6  *
7  * @param <T> type des elements
8  */
```

```

9  public interface ExtrinsicMinPQ<T> {
10
11     /** Ajoute l'element avec la priorite donnee.
12     * @throws IllegalArgumentException si item est deja present. */
13     void add(T item, double priority);
14
15     /** @return true si l'element est dans la file. */
16     boolean contains(T item);
17
18     /** @return l'element de plus petite priorite (sans le retirer).
19     * @throws NoSuchElementException si la file est vide. */
20     T peekMin();
21
22     /** Retire et renvoie l'element de plus petite priorite.
23     * @throws NoSuchElementException si la file est vide. */
24     T removeMin();
25
26     /** Modifie la priorite d'un element deja present.
27     * @throws NoSuchElementException si item n'est pas present. */
28     void changePriority(T item, double priority);
29
30     /** @return le nombre d'elements dans la file. */
31     int size();
32 }

```

2.2 La classe OptimizedHeapMinPQ<T> — à implémenter

```

1  import java.util.*;
2
3  public class OptimizedHeapMinPQ<T> implements ExtrinsicMinPQ<T> {
4
5     /** Paire (element, priorite) stockee dans le tableau. */
6     private static class Node<T> {
7         T item;
8         double priority;
9         Node(T i, double p) { this.item = i; this.priority = p; }
10    }
11
12    // Le tableau du heap : indice 0 = racine = minimum
13    private final List<Node<T>> heap;
14
15    // La table de localisation : item -> indice courant dans heap
16    private final Map<T, Integer> itemToIndex;
17
18    public OptimizedHeapMinPQ() {
19        this.heap = new ArrayList<>();
20        this.itemToIndex = new HashMap<>();
21    }
22
23    // A VOUS : add, contains, peekMin, removeMin, changePriority, size
24 }

```

3 Ce qu'il faut faire — étape par étape

3.1 Étape 1 — Mettre en place le projet

Dans le conteneur Docker :

```
new-project P3-Heap
cd P3-Heap
```

Créez les fichiers :

- `src/main/java/ExtrinsicMinPQ.java` — copier l'interface fournie.
- `src/main/java/OptimizedHeapMinPQ.java` — votre implémentation.
- `src/test/java/OptimizedHeapMinPQTest.java` — tests (fournis + à compléter).

3.2 Étape 2 — Implémenter `add` et la percolation vers le haut

Commencez par faire fonctionner l'insertion :

1. Vérifier que l'élément n'est pas déjà présent (lever `IllegalArgumentException`).
2. Ajouter la paire à la fin de `heap`.
3. Mettre à jour `itemToIndex`.
4. Appeler `percolateUp` sur le dernier indice.

Piste sans solution

La percolation vers le haut ressemble à celle vue en CM, mais chaque swap doit maintenant échanger *deux entrées* dans `itemToIndex`. Écrivez d'abord une méthode `swap(int i, int j)` qui fait les quatre mises à jour d'un coup. C'est la méthode la plus importante du projet.

3.3 Étape 3 — Implémenter `peekMin`, `removeMin`, `contains`, `size`

- `peekMin` : l'élément à l'indice 0 (avec gestion du cas vide).
- `removeMin` : noter l'élément à l'indice 0, le remplacer par le dernier élément, réduire la taille, mettre à jour `itemToIndex` (supprimer l'ancien min, relocaliser le nouveau), puis `percolateDown` sur l'indice 0.
- `contains` : une simple interrogation de la `HashMap`.
- `size` : la taille du tableau.

3.4 Étape 4 — Implémenter `changePriority`

C'est l'opération qui fait toute la valeur du projet.

Piste sans solution

Algorithme :

1. Récupérer l'indice actuel de `item` via `itemToIndex` — $\mathcal{O}(1)$.
2. Comparer l'ancienne et la nouvelle priorité.
3. Si la nouvelle est *plus petite*, potentiellement `percolateUp`.
4. Si la nouvelle est *plus grande*, potentiellement `percolateDown`.
5. Si les deux sont égales, ne rien faire.

Chaque percolation est en $\mathcal{O}(\log n)$. Le *lookup* initial est en $\mathcal{O}(1)$ amorti grâce à la `HashMap`.

Total : $\mathcal{O}(\log n)$ amorti.

4 Tests JUnit

4.1 Tests fournis (à copier tels quels)

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 public class OptimizedHeapMinPQTest {
```

```
5
6  @Test
7  void testAddPeek() {
8      OptimizedHeapMinPQ<String> pq = new OptimizedHeapMinPQ<>();
9      pq.add("Dakar", 3.0);
10     pq.add("Thies", 1.5);
11     pq.add("Saint-Louis", 2.7);
12     assertEquals("Thies", pq.peekMin());
13     assertEquals(3, pq.size());
14 }
15
16 @Test
17 void testRemoveMinOrdreCroissant() {
18     OptimizedHeapMinPQ<Integer> pq = new OptimizedHeapMinPQ<>();
19     int[] donnees = {5, 2, 8, 1, 9, 3, 7};
20     for (int x : donnees) pq.add(x, x);
21     int[] attendu = {1, 2, 3, 5, 7, 8, 9};
22     for (int exp : attendu) assertEquals(exp, pq.removeMin());
23     assertEquals(0, pq.size());
24 }
25
26 @Test
27 void testAddDejaPresentLeveException() {
28     OptimizedHeapMinPQ<String> pq = new OptimizedHeapMinPQ<>();
29     pq.add("Dakar", 1.0);
30     assertThrows(IllegalArgumentException.class, () -> pq.add("Dakar",
31         2.0));
32 }
33
34 @Test
35 void testChangePrioritySmaller() {
36     OptimizedHeapMinPQ<String> pq = new OptimizedHeapMinPQ<>();
37     pq.add("A", 10.0);
38     pq.add("B", 20.0);
39     pq.add("C", 30.0);
40     assertEquals("A", pq.peekMin());
41     pq.changePriority("C", 1.0);
42     assertEquals("C", pq.peekMin());
43 }
44
45 @Test
46 void testChangePriorityLarger() {
47     OptimizedHeapMinPQ<String> pq = new OptimizedHeapMinPQ<>();
48     pq.add("A", 10.0);
49     pq.add("B", 20.0);
50     pq.add("C", 30.0);
51     pq.changePriority("A", 100.0);
52     assertEquals("B", pq.peekMin());
53 }
54
55 @Test
56 void testContains() {
57     OptimizedHeapMinPQ<Integer> pq = new OptimizedHeapMinPQ<>();
58     pq.add(42, 1.0);
59     assertTrue(pq.contains(42));
60     assertFalse(pq.contains(99));
61     pq.removeMin();
62     assertFalse(pq.contains(42));
63 }
64
65 @Test
```

```

65     void testPeekVideLeveException() {
66         OptimizedHeapMinPQ<Integer> pq = new OptimizedHeapMinPQ<>();
67         assertThrows(RuntimeException.class, pq::peekMin);
68     }
69 }

```

4.2 Tests à écrire (au moins 3)

Écrivez au moins 3 tests supplémentaires qui couvrent :

- **Un test de stress** — ajouter 1 000 éléments dans un ordre aléatoire, les retirer un par un et vérifier qu'ils sortent dans l'ordre croissant.
- **Un test mixte** — intercaler `add`, `removeMin` et `changePriority` et vérifier la cohérence à chaque étape.
- **Un test de `changePriority` en cascade** — modifier la priorité du minimum pour qu'il descende jusqu'à une feuille (plusieurs swaps consécutifs).

5 Benchmark comparatif

Dans un fichier `Main.java`, comparez empiriquement votre `OptimizedHeapMinPQ` avec `java.util.PriorityQueue` sur un scénario qui *force* l'utilisation de `changePriority`.

Scénario suggéré

1. Insérer n éléments avec des priorités aléatoires ($n = 10\,000$).
 2. Effectuer n opérations `changePriority` sur des éléments aléatoires (avec la `PriorityQueue` standard, simuler par `remove` + `add`).
 3. Mesurer le temps total avec `System.nanoTime()` pour chaque structure.
 4. Répéter pour $n = 1\,000$, $10\,000$, $100\,000$ et tracer la courbe.
- Vous devriez observer un écart croissant. Documentez vos résultats dans le rapport.

6 Rapport — contenu attendu (3 à 5 pages)

Le rapport doit contenir :

1. **Introduction (1/2 page)** — rappeler le besoin de `changePriority` et le contexte de Dijkstra.
2. **Analyse de complexité (1 page)** — tableau des complexités de chaque opération, *avec justification* :

Opération	Complexité	Justification
<code>add</code>	...	...
<code>peekMin</code>	...	...
<code>removeMin</code>	...	...
<code>changePriority</code>	...	...
<code>contains</code>	...	...
<code>size</code>	...	...

3. **Benchmark (1 à 2 pages)** — tableau ou graphique montrant le temps d'exécution, et interprétation.
4. **Discussion sur la HashMap interne (1/2 page)** — pourquoi `HashMap` et pas `TreeMap`? Quel impact du facteur de charge? Que se passerait-il avec une `ArrayList`?
5. **Conclusion (1/4 page)** — ce que vous avez retenu, difficultés rencontrées.

7 Grille d'évaluation

Critère	Points	Détail
Compilation et exécution	2 pts	Le code compile, les tests fournis passent.
<code>add</code> + <code>peekMin</code> + <code>size</code>	2 pts	Implémentation correcte, tests passent.
<code>removeMin</code>	3 pts	Avec mise à jour correcte de <code>itemToIndex</code> .
<code>changePriority</code>	4 pts	Les deux sens (up et down), percolation correcte.
<code>contains</code>	1 pt	Utilisation efficace de la <code>HashMap</code> .
Tests supplémentaires (3+)	2 pts	Pertinents, non redondants avec les tests fournis.
Benchmark vs <code>PriorityQueue</code>	2 pts	Exécution, mesures, interprétation.
Analyse de complexité	2 pts	Tableau justifié dans le rapport.
Qualité du rapport	2 pts	Clarté, structure, langue.
Total	20 pts	

Bonus (jusqu'à +2 pts) :

- Heap à 4 enfants au lieu de 2 — discuter le gain (CSE 373 le fait).
- Optimisation mémoire — capacité initiale paramétrable, redimensionnement dynamique documenté.
- Intégration avec un mini-Dijkstra sur un petit graphe pour montrer l'usage réel (anticipation de S05).

8 Règles et conseils

Règles strictes

- Projet **individuel**. Toute similitude suspecte entre deux rendus fera l'objet d'une convocation et d'une re-soutenance orale.
- Vous pouvez consulter tout document, mais **le code doit être le vôtre**. Citez vos sources dans le rapport.
- L'usage de `java.util.PriorityQueue` comme structure interne **est interdit** (sauf dans le benchmark où on la compare).
- Il est **interdit** de chercher un élément par parcours linéaire dans `changePriority` — si vous le faites, la complexité attendue n'est pas atteinte et la note est fortement réduite.

Conseils pratiques :

- Écrivez `swap(i, j)` dès le début et utilisez-la partout. La plupart des bugs de ce projet viennent d'une mise à jour oubliée de `itemToIndex`.
- Faites tourner les tests après chaque méthode implémentée, pas à la fin.
- Pour le benchmark, prenez soin de *préchauffer* la JVM (exécution factice avant les mesures)
 - Java optimise au fur et à mesure.
- Commitez votre code régulièrement avec Git.

9 Remise et format du rendu

Format du rendu

1. Une archive `P3-Heap-PRENOM-NOM.zip` contenant :
 - Le dossier du projet Gradle (sans `build/` ni `.gradle/`).
 - Le rapport en PDF à la racine.
 - Un fichier `README.md` avec votre nom, la commande pour lancer les tests, et toute remarque utile.
2. Ou bien : un lien vers un dépôt Git public contenant la même arborescence.

Références utiles.

- Sedgewick & Wayne, *Algorithms* 4e — §2.4 Priority Queues.
- CLRS 4e — Chapitre 6 Heapsort, section 6.5 Priority Queues (*heap-decrease-key*).
- Cours CSE 373 (University of Washington) — LEC 12 Heap Implementation.